Technical specification: cubic PH spline family

1. Scope and mathematical limitations

`CubicPHSplineOpen` represents an **open planar spline of cubic PH segments interpolating arbitrary admissible input points** - convex, collinear, or general free-form data with inflections and mixed straight and curved spans.

`CubicPHSplineClosed` supplies the corresponding cyclic topology. Section 23 defines its square convex G^2 system, seam contract, and exact reuse of the general-data auxiliary-inflection machinery. `CubicPHSpline` is their abstract family base and represents neither topology by itself.

For input points

$$P_0, P_1, \dots, P_m \in \mathbb{R}^2,$$

the spline contains one cubic PH segment per input span

$$P_i \longrightarrow P_{i+1}, \qquad i = 0, \dots, m-1,$$

except that each *inflection span* (section 22) carries exactly two segments joined at one **auxiliary inflection point** inserted by the deterministic recipe of section 22.

A non-straight regular planar cubic PH segment has no inflection, so a globally G^2 spline of such segments cannot change the sign of curvature: this is a mathematical fact, not an implementation choice. The published construction (Jaklič, Kozak, Krajnc, Vitrih, Žagar, *On interpolation by planar cubic G^2 Pythagorean-hodograph spline curves*, Math. Comp.) resolves it by a preprocessing algorithm (its Section 6): whenever the data polygon has an inflection, one additional point with a prescribed tangent direction is inserted there, and continuity at that point is reduced from G^2 to G^1. This specification adopts that recipe as part of the class contract.

The implementation shall enforce:

point interpolation + G^2 within convex sub-splines
+ G^1 at inserted inflection points and straight/curved transitions
+ regular cubic PH segments
+ exact rational NURBS offsets
+ elementary local arc-length inversion.

Accordingly, the constructor shall:

- accept arbitrary finite planar point sequences subject to the validity rules of section 6;
- interpolate strictly convex, consistently oriented sequences with a globally G^2 spline (no auxiliary points);
- interpolate a completely collinear, monotonically ordered sequence as a degenerate straight spline;

- for general data, partition the sequence at curvature sign changes and insert exactly one auxiliary inflection point per inflection span, following the mathematically defined recipe of section 22 - never any other point, never at any other location;
- guarantee exact G^1 (common unit tangent) at every auxiliary inflection point and at every straight/curved transition, and full G^2 everywhere else;
- reject reversals, duplicate points within a convex sub-polyline, and self-intersections within a convex sub-polyline;
- never downgrade continuity anywhere except at the documented G^1 joints above;
- expose on-demand construction of an exact rational NURBS representation of every representable finite signed parallel offset, without sampling or geometric fitting;
- never return an approximate solution whose verified continuity mismatch exceeds the construction tolerances.

The underlying interpolation problem on each convex sub-spline is a nonlinear tridiagonal system for the unknown internal tangent directions. Under the published convexity and angle restrictions, an admissible solution exists, and under a slightly stronger angle bound it is unique. (reference)

2. Public API

`CubicPHSpline` is the abstract cubic-family base and SHALL NOT be directly instantiable. `CubicPHSplineOpen` and `CubicPHSplineClosed` SHALL be direct sibling subclasses of that base; neither concrete topology class may inherit from the other. Both remain subclasses of the package-wide `PHSpline` base.

2.1 Constructor

`CubicPHSplineOpen(points)`

Accepted input:

`points: list[list[Real] | tuple[Real, Real]]`

Requirements:

- the outer object must be a finite list;
- every element must be a list or tuple of exactly two real coordinates;
- coordinates must be finite;
- Boolean coordinates are rejected;
- at least two points are required;
- the constructor copies the data and retains no references to mutable input objects.

The resulting object shall be immutable from the public API.

The closed constructor is:

`CubicPHSplineClosed(points)`

It accepts at least three cyclic points. Each authoritative point SHALL be listed once: an exactly repeated final seam point is rejected. The constructor SHALL close the final point back to the first internally and SHALL verify the declared seam continuity independently. Section 23 defines its cyclic construction and the additional admissibility rules.

2.2 Required methods

```
curve.point(u)
curve.tangent(u)
curve.normal(u, side="left")
curve.principal_normal(u)
curve.signed_curvature(u)
```

```
curve.curvature_vector(u)

curve.arc_length(u)
curve.parameter_at_length(s)
curve.point_at_length(s)

curve.offset(distance)
```

2.3 Return types

Method	Return value
`point`	NumPy `float64` array of shape `(2,)`
`tangent`	NumPy `float64` unit vector of shape `(2,)`
`normal`	NumPy `float64` unit vector of shape `(2,)`
`principal_normal`	NumPy `float64` unit vector of shape `(2,)`
`signed_curvature`	Python `float`
`curvature_vector`	NumPy `float64` vector of shape `(2,)`
`arc_length`	Python `float`
`parameter_at_length`	Python `float`
`point_at_length`	NumPy `float64` array of shape `(2,)`
`offset`	immutable `NURBSHandle`

All returned arrays shall be newly allocated or read-only. A caller must not be able to mutate the spline through a returned object.

2.4 Read-only NURBS handle

`curve.offset(distance)` SHALL return the same public `NURBSHandle` type for the cubic and PH B-spline families. The handle has only the following inspection and point-query interface:

```
class NURBSHandle:
    @property
    def degree(self) -> int: ...

    @property
    def knots(self) -> NDArray[np.float64]: ...

    @property
    def control_points(self) -> NDArray[np.float64]: ...

    @property
    def weights(self) -> NDArray[np.float64]: ...

    @property
    def num_control_points(self) -> int: ...

    @property
    def num_spans(self) -> int: ...

    @property
    def domain(self) -> tuple[float, float]: ...

    @property
```

```
    def closed(self) -> bool: ...

    def point(self, u: Real) -> NDArray[np.float64]: ...
```

The interface SHALL NOT expose mutation, editing, derivatives, arc-length queries, fitting controls, or a back-reference that can mutate the source spline. `domain` is exactly `(0.0, 1.0)`. `knots` has shape `(num_control_points + degree + 1,)`, `control_points` has shape `(num_control_points, 2)`, and `weights` has shape `(num_control_points,)`. All three arrays contain finite binary64 values and are returned as read-only snapshots. Every weight is strictly positive.

The handle is a snapshot. It remains valid and unchanged even if the source is a mutable `PHBSpline` that is edited later. Section 11.7 defines the normative construction and evaluation rules.

3. Public parameter convention

Let

$$m = \text{len}(p) - 1.$$

The global public parameter is

$$u \in [0, 1].$$

Use uniform global knots:

$$u_i = \frac{i}{m}, \qquad i = 0, \dots, m.$$

Thus:

$$\boxed{\text{curve.}\,\text{point}(u_i) = P_i.}$$

For $0 \le u < 1$, define

$$q = mu, \qquad i = \lfloor q \rfloor, \qquad t = q - i.$$

Here:

- i is the input-span index;
- $t \in [0, 1)$ is that span's local parameter.

At $u = 1$, use

$$i = m - 1, \qquad t = 1.$$

On an ordinary span, t is the segment's local polynomial parameter. On an inflection span carrying two segments joined at the auxiliary point P' with chord fraction $\rho \in (0, 1)$ (section 22), the span parameter is subdivided proportionally to the chord split: local parameters $t \in [0, \rho)$ map to the first segment at t/ρ, and $t \in [\rho, 1]$ map to the second segment at $(t-\rho)/(1-\rho)$. The auxiliary point therefore sits at the internal knot

$$u_{P'} = \frac{i + \rho}{m},$$

which is **not** one of the public knots u_i.

Interior knots are assigned to the segment on their right. At G^2 joins this convention is unobservable because point, tangent and curvature are all continuous there. At an auxiliary inflection point and at a straight/curved transition, point and tangent are continuous but signed curvature jumps (section 22); the right-sided convention then determines which one-sided curvature value the evaluation methods return at the exact knot.

The global parameter u is **not** arc length. Consequently:

- `point(u)` is not necessarily parametrically C^1 or C^2;
- the geometric spline is G^2 within convex sub-splines and exactly G^1 at the joints of section 22;
- `point_at_length(s)` is C^2 across all G^2 spline joins and C^1 at the G^1 joints (section 14.4).

4. Segment representation

Each segment shall be represented internally in both cubic Bezier form and PH preimage form.

4.1 Bezier form

For segment i, joining P_i to P_{i+1}, store

$$B_0 = P_i,$$

$$B_1 = P_i + \lambda_0 d_i,$$

$$B_2 = P_{i+1} - \lambda_1 d_{i+1},$$

$$B_3 = P_{i+1},$$

where:

- d_i, d_{i+1} are unit tangent directions;
- $\lambda_0, \lambda_1 > 0$ are Bezier edge lengths determined by the cubic-PH condition.

Point evaluation shall use de Casteljau evaluation, not naive expansion of the cubic power basis.

4.2 PH preimage form

Represent planar vectors as complex numbers internally or as equivalent two-component real vectors.

For local parameter t,

$$z_i'(t) = w_i(t)^2,$$

where

$$w_i(t) = (1-t)w_{i,0} + tw_{i,1}.$$

Define

$$a = w_{i,0}, \qquad b = w_{i,1} - w_{i,0}.$$

Then

$$w_i(t) = a + bt.$$

The endpoint preimages are

$$w_{i,0} = \sqrt{3\lambda_0}\, e^{\frac{1}{2} i\theta_i},$$

$$w_{i,1} = \sqrt{3\lambda_1}\, e^{\frac{1}{2} i\theta_{i+1}},$$

where θ_i and θ_{i+1} are consistently unwrapped tangent angles.

The implementation shall verify after construction that

$$\frac{w_{i,0}^2 + w_{i,0} w_{i,1} + w_{i,1}^2}{3}$$

agrees with

$$P_{i+1} - P_i$$

within the normalized construction tolerance.

4.3 Stored scalar invariants

For each segment store

$$A = |b|^2,$$

$$B = 2\,\mathrm{Re}(\overline{a}b),$$

$$C = |a|^2,$$

and

$$\chi = \mathrm{Im}(\overline{a}b).$$

Then the local speed is

$$\boxed{\sigma(t) = |z_i'(t)| = |a + bt|^2 = At^2 + Bt + C.}$$

The signed curvature is

$$\boxed{\kappa(t) = \frac{2\chi}{\sigma(t)^2}.}$$

The sign of χ is constant on a segment. Within one curved convex sub-spline all segments have the same sign of χ; the sign flips exactly at auxiliary inflection points, and $\chi = 0$ exactly on straight segments. The sign pattern of χ along the segment list therefore reproduces the convex partition of section 6.3 and shall be verified against it.

5. Internal coordinate normalization

All construction shall be performed in normalized coordinates.

Choose:

$$O = P_0,$$

and

$$H = \max_i \|P_{i+1} - P_i\|.$$

Construct normalized points

$$\widehat{P}_i = \frac{P_i - O}{H}.$$

Requirements:

- H must be finite and strictly positive;
- subtraction and scaling must produce finite values;
- if any normalized chord vanishes, construction fails;
- if the shortest chord is too small to be distinguishable relative to the longest chord in binary64 arithmetic, construction fails.

Recommended minimum chord-ratio requirement:

$$\frac{\min_i \|\Delta P_i\|}{\max_i \|\Delta P_i\|} > 1024\,\varepsilon_{\text{mach}}.$$

All PH coefficients, nonlinear solving and regularity tests use normalized coordinates.

Conversion back to original coordinates is:

$$P = O + H\widehat{P}.$$

Curvature transforms as

$$\kappa = \frac{\hat{\kappa}}{H}.$$

Arc length transforms as

$$s = H\hat{s}.$$

Tangents and unit normals are unchanged by positive uniform scaling.

The implementation shall use `hypot`-style norm calculations. It shall not compute norms using an unscaled expression such as $x^2 + y^2$ when overflow or underflow is possible.

6. Input classification

6.1 Chords

Compute

$$\Delta_i = P_{i+1} - P_i, \qquad \ell_i = \|\Delta_i\|.$$

Every ℓ_i must be finite and nonzero.

Consecutive duplicate points produce `DegeneratePointDataError`.

Nonconsecutive duplicate points produce `NonSimplePointDataError` when they fall within one convex sub-polyline (section 6.3); duplicates belonging to different convex sub-polylines are permitted, since general free-form data may revisit a location.

6.2 Collinear case

Define normalized turn measures

$$c_i = \frac{\Delta_{i-1} \times \Delta_i}{\|\Delta_{i-1}\| \, \|\Delta_i\|}.$$

If every c_i is numerically zero, the sequence is classified as collinear.

For a collinear sequence, project all chords onto the first chord direction. Every projected chord length must be strictly positive. Thus the points must proceed monotonically along the line without reversing.

A valid collinear sequence is represented as one degenerate straight cubic PH segment per input span:

$$w_{i,0} = w_{i,1},$$

and

$$z_i(t) = P_i + t(P_{i+1} - P_i).$$

This spline is geometrically G^∞.

If collinearity is mixed - some turns are numerically zero while others are materially nonzero - the maximal collinear sub-sequences form degenerate straight sub-splines, joined to their curved neighbours with exact G^1 continuity at the shared input points (section 22.5). A curved cubic PH segment cannot join a straight segment with G^2: the curvature jumps between zero and a nonzero value there, and this jump is part of the documented contract, never silent.

6.3 General curved case and convex partition

Define the sign sequence of the material cross products

$$s_i = \operatorname{sign}(\Delta_{i-1} \times \Delta_i).$$

The sequence is partitioned into **maximal convex sub-sequences** on which s_i is constant; each carries its own orientation

$$\tau \in \{-1, +1\} :$$

- $\tau = +1$: counterclockwise or left-turning;
- $\tau = -1$: clockwise or right-turning.

A sign change between two consecutive material turns identifies an **inflection span**: the input span whose two bounding interior points turn in opposite directions. Each inflection span receives exactly one auxiliary inflection point by the recipe of section 22; the resulting sub-splines join there with exact G^1 continuity and a curvature sign change.

A turn angle of approximately π produces `ReversalError`.

Each convex sub-polyline (including its auxiliary endpoints, section 22) shall be checked for proper intersections between its nonadjacent chord segments; any such intersection produces `NonSimplePointDataError`. Chords belonging to different convex sub-polylines may intersect freely: general free-form data is allowed to produce a self-intersecting spline. Nonconsecutive duplicate points are likewise rejected only within one convex sub-polyline.

7. Boundary tangent policy

Every convex sub-spline is an independent interpolation problem whose two boundary tangent directions must be determined. There are two cases:

1. **Free boundaries** - the global spline start and end. Points alone do not determine these tangents, so the constructor uses the deterministic local-circumcircle policy of this section, including the minimal clamp of section 7.2.
2. **Prescribed boundaries** - auxiliary inflection points and straight/curved transition points. There the tangent direction is fixed by section 22 (G^1 requires both adjacent sub-splines to share it exactly), so no policy freedom exists and **no clamping is permitted**: if a prescribed boundary deviation violates the uniqueness condition of section 7.2, the constructor raises `InterpolationDomainError`.

This section specifies the free-boundary circumcircle policy.

7.1 Initial endpoint tangent

For the first three points P_0, P_1, P_2, compute their circumcenter O_0. Define the raw start tangent as

$$d_0^{\text{raw}} = \tau J(P_0 - O_0),$$

where

$$J(x, y) = (-y, x).$$

Normalize it and orient it so that

$$d_0^{\text{raw}} \cdot (P_1 - P_0) > 0.$$

For the last three points, compute the circumcenter O_m and use

$$d_m^{\text{raw}} = \tau J(P_m - O_m),$$

again oriented in the direction of traversal.

The circumcenter calculation must be performed in normalized coordinates and must reject a determinant whose magnitude is insufficient for reliable computation.

7.2 Boundary-angle adjustment

Let ψ_i be consistently unwrapped chord angles. Define the positive interior turns

$$\phi_i = \tau(\psi_i - \psi_{i-1}), \qquad i = 1, \dots, m-1.$$

Define raw boundary turns:

$$\phi_0^{\text{raw}} = \tau(\psi_0 - \theta_0^{\text{raw}}),$$

$$\phi_m^{\text{raw}} = \tau(\theta_m^{\text{raw}} - \psi_{m-1}).$$

The unique-admissible-solution bound is

$$\Theta_{\text{unique}} = \pi + \arccos\frac{1}{\sqrt{3}} \approx 4.096909271714303.$$

The published uniqueness condition is

$$\phi_i + \phi_{i+1} < \Theta_{\text{unique}}.$$

(reference)

Interior violations cannot be repaired without adding points, so they produce `InterpolationDomainError`.

Boundary tangent deviations may be minimally clamped:

$$\phi_0 = \min\left(\phi_0^{\text{raw}}, \Theta_{\text{unique}} - \phi_1 - \delta_\theta\right),$$

$$\phi_m = \min\left(\phi_m^{\text{raw}}, \Theta_{\text{unique}} - \phi_{m-1} - \delta_\theta\right).$$

Use an angular safety margin such as

$$\delta_\theta = \max(1024\varepsilon_{\text{mach}}, 10^{-12})$$

radians.

The final boundary angles are

$$\theta_0 = \psi_0 - \tau\phi_0,$$

$$\theta_m = \psi_{m-1} + \tau\phi_m.$$

The constructor shall record internally whether either raw estimate was clamped.

8. Cubic PH segment construction from tangent directions

Given:

- endpoints P_0, P_1;
- chord length ℓ;
- chord angle ψ;
- tangent angles θ_0, θ_1;

define positive oriented endpoint deviations

$$\alpha = \tau(\psi - \theta_0),$$

$$\beta = \tau(\theta_1 - \psi).$$

Require

$$\alpha > 0, \qquad \beta > 0.$$

Define

$$\beta_0 = \frac{\beta - \alpha}{2}, \qquad \beta_1 = \frac{\alpha + \beta}{2}.$$

Then

$$\xi_0 = \frac{1}{2}\frac{\sin\beta_0}{\sin\beta_1}.$$

For small arguments, the ratio shall be evaluated using a `sinc`-based form:

$$\frac{\sin\beta_0}{\sin\beta_1} = \frac{\beta_0}{\beta_1}\frac{\operatorname{sinc}\beta_0}{\operatorname{sinc}\beta_1}.$$

Define

$$q = \cos(2\beta_1),$$

$$D = 2\cos\beta_0\cos\beta_1,$$

$$A_q = 1 + 2q,$$

$$C_q = 1 - (1 - 2q)\xi_0^2,$$

and

$$\Delta_q = D^2 - A_q C_q.$$

The discriminant must satisfy

$$\Delta_q \geq 0$$

within floating-point tolerance.

A small negative value caused solely by rounding may be replaced by zero. A materially negative value is a construction failure.

Use the rationalized, cancellation-resistant admissible root:

$$\boxed{\xi_1 = \frac{C_q}{D + \sqrt{\Delta_q}}.}$$

Do not use the subtraction-prone equivalent

$$\frac{D - \sqrt{\Delta_q}}{A_q}.$$

Then

$$\boxed{\lambda_0 = \ell(\xi_1 + \xi_0), \qquad \lambda_1 = \ell(\xi_1 - \xi_0).}$$

Both must be finite and strictly positive.

A materially nonpositive λ_j is an invalid segment, not a value to clamp.

9. Solving the global G^2 system

9.1 Unknown tangent angles

For each internal point P_i, $i = 1, \ldots, m-1$, its tangent must lie strictly inside the wedge between the adjacent chord directions.

Use normalized scalar variables

$$x_i \in (0, 1)$$

defined by

$$\tau\theta_i = (1 - x_i)\tau\psi_{i-1} + x_i\tau\psi_i.$$

This bounded representation prevents tangent directions from leaving their admissible cones.

Numerical bounds shall be

$$x_{\min} \leq x_i \leq 1 - x_{\min},$$

where

$$x_{\min} = 64\varepsilon_{\text{mach}}$$

or a slightly larger implementation constant.

9.2 Endpoint curvatures of one segment

For a segment with edge lengths λ_0, λ_1 and positive oriented tangent turn

$$\delta = \tau(\theta_1 - \theta_0) > 0,$$

the positive curvature magnitudes at its start and end are

$$k^{(0)} = \frac{2}{3} \sin \frac{\delta}{2} \sqrt{\frac{\lambda_1}{\lambda_0^3}},$$

$$k^{(1)} = \frac{2}{3} \sin \frac{\delta}{2} \sqrt{\frac{\lambda_0}{\lambda_1^3}}.$$

The signed values are

$$\kappa^{(0)} = \tau k^{(0)}, \qquad \kappa^{(1)} = \tau k^{(1)}.$$

For numerical stability, compute logarithms:

$$\log k^{(0)} = \log \frac{2}{3} + \log \sin \frac{\delta}{2} + \frac{1}{2} \log \lambda_1 - \frac{3}{2} \log \lambda_0,$$

$$\log k^{(1)} = \log \frac{2}{3} + \log \sin \frac{\delta}{2} + \frac{1}{2} \log \lambda_0 - \frac{3}{2} \log \lambda_1.$$

9.3 Residual system

At internal point P_i, require equal curvature from the adjacent segments:

$$\kappa_{i-1}^{(1)} = \kappa_i^{(0)}.$$

Use the dimensionless residual

$$\boxed{F_i(x) = \log k_{i-1}^{(1)} - \log k_i^{(0)}.}$$

The solution satisfies

$$F_i(x) = 0, \qquad i = 1, \dots, m-1.$$

Each F_i depends only on

$$x_{i-1}, x_i, x_{i+1},$$

with boundary angles treated as fixed. Therefore the Jacobian is tridiagonal.

For general data the curvature-continuity equations are posed per convex sub-spline; boundary angles of every sub-spline (free, clamped, or prescribed by section 22) are fixed before the solve. The global system therefore **decouples into independent tridiagonal blocks**, one per curved convex sub-spline, which are solved and accepted independently. A curved sub-spline with no interior input point (both tangents prescribed) has an empty system: its single segment is evaluated directly by section 8 and still passes the full acceptance battery.

9.4 Initial estimate

Use the centered secant direction

$$d_i^{(0)} = \frac{P_{i+1} - P_{i-1}}{\|P_{i+1} - P_{i-1}\|}$$

as the raw initial internal tangent, then project its angle strictly into the admissible wedge.

This is also the initialization proposed in the numerical construction from the cubic-PH G^2 interpolation literature. (reference)

9.5 Required solver

Use a bounded trust-region nonlinear least-squares or bounded trust-region root solver.

Required properties:

- all iterates remain inside the tangent-angle box;
- invalid segment parameters are never accepted;
- step rejection is supported;
- the tridiagonal Jacobian structure is exploited;
- the solve is deterministic;
- no random initialization is used;
- solver success flags are not accepted without independent post-verification.

Recommended implementation:

- SciPy trust-region reflective least squares;
- dimensionless logarithmic curvature residuals;
- Jacobian sparsity corresponding to the tridiagonal structure;
- Jacobian scaling enabled.

The Jacobian shall be computed using either:

1. analytic differentiation; or
2. complex-step differentiation of a branch-free smooth residual core.

Ordinary forward finite differences should not be the sole production Jacobian because they lose accuracy precisely in the small-angle and highly unequal-span cases where the construction is most sensitive.

A damped version of the published fixed-point iteration may be used only as an initializer. Its convergence is not proven generally, so it shall not be the sole construction method. (reference)

9.6 Solver acceptance

The solver result is accepted only if all of the following hold:

$$\max_i |F_i| \le 10^{-11},$$

all tangent variables lie strictly inside their wedges,

$$\lambda_{i,0} > 0, \qquad \lambda_{i,1} > 0,$$

every PH segment is regular,

and all segment reconstruction residuals satisfy the normalized geometric tolerance.

If any condition fails, raise `SplineConvergenceError`.

Never return the last nonlinear iterate as an approximate spline.

10. Regularity and admissibility checks

10.1 Minimum speed

For

$$\sigma(t) = At^2 + Bt + C,$$

the minimizing parameter is

$$t_* = \operatorname{clamp}\left(-\frac{B}{2A}, 0, 1\right)$$

when $A > 0$.

For $A = 0$, the speed is constant.

Compute

$$\sigma_{\min} = \sigma(t_*).$$

Require

$$\sigma_{\min} > 0.$$

Also require a relative regularity margin:

$$\frac{\sigma_{\min}}{\max(\sigma(0), \sigma(1))} > \rho_{\min},$$

with a recommended default

$$\rho_{\min} = 10^{-12}.$$

A segment that violates this condition is nearly cuspidal and shall produce `NonRegularSplineError`.

No method shall divide by a speed before this regularity condition has been verified.

10.2 Control-polygon admissibility

For each Bezier segment require, in the spline orientation,

$$\tau(\Delta B_0 \times \Delta B_1) > 0,$$

$$\tau(\Delta B_1 \times \Delta B_2) > 0.$$

A materially nonpositive result produces `NonAdmissibleSegmentError`.

10.3 Global continuity verification

Internal joints are of two kinds and are verified independently after construction:

- G^2 **joints**: every joint interior to a convex sub-spline;
- G^1 **joints**: every auxiliary inflection point and every straight/curved transition (section 22).

At every G^1 joint verify tangent continuity

$$\|T_- - T_+\| \leq \varepsilon_T$$

and, at an auxiliary inflection point, the curvature sign change

$$\kappa_- \, \kappa_+ < 0.$$

At a straight/curved transition verify that the straight side reports exactly zero signed curvature. No bound is imposed on the curvature jump magnitude at G^1 joints; the jump is the documented cost of the inflection.

At every G^2 joint verify independently:

$$\|T_{i-1}(1) - T_i(0)\| \leq \varepsilon_T,$$

and

$$\frac{|\kappa_{i-1}(1) - \kappa_i(0)|}{\max(|\kappa_{i-1}(1)|, |\kappa_i(0)|, H^{-1}\varepsilon_{\text{mach}})} \leq \varepsilon_\kappa.$$

Recommended values:

$$\varepsilon_T = 10^{-12}, \qquad \varepsilon_\kappa = 10^{-10}.$$

Failure produces `G2VerificationError`.

11. Geometric evaluation

11.1 `point(u)`

1. Validate u.
2. Locate segment i and local parameter t.
3. At an exact knot, return the stored original input point.
4. Otherwise evaluate the normalized cubic with de Casteljau.
5. Transform back to original coordinates.
6. Verify that the result is finite.

11.2 `tangent(u)`

Evaluate

$$w(t) = a + bt.$$

Normalize w:

$$\widehat{w} = \frac{w}{|w|}.$$

Then

$$\boxed{T(t) = \widehat{w}^2.}$$

In real components, if

$$\widehat{w} = (r, s),$$

then

$$T = (r^2 - s^2, 2rs).$$

This form is preferred over separately forming w^2 and dividing by $|w|^2$, because it reduces overflow and normalization error.

The output shall be renormalized only if its norm differs from one by more than a small multiple of machine epsilon.

11.3 `normal(u, side="left")`

For

$$T = (T_x, T_y),$$

the left unit normal is

$$N_L = (-T_y, T_x).$$

The right unit normal is

$$N_R = -N_L.$$

Accepted values for `side` are exactly:

`"left"`
`"right"`

Any other value produces `ValueError`.

The default is `"left"`.

11.4 `signed_curvature(u)`

Evaluate

$$\sigma(t) = |w(t)|^2.$$

Then

$$\widehat{\kappa}(t) = \frac{2\chi}{\sigma(t)^2}$$

in normalized coordinates, and

$$\boxed{\kappa(t) = \frac{\widehat{\kappa}(t)}{H}}$$

in user coordinates.

The method shall not infer curvature from finite differences of evaluated points.

11.5 `principal_normal(u)`

The principal normal is

$$N_P = \operatorname{sign}(\kappa) N_L.$$

Equivalently, it points toward the local center of curvature.

On curved segments $\kappa \neq 0$ pointwise (the segment χ is nonzero), so the principal normal is defined; for general data its side flips across auxiliary inflection points together with the curvature sign. At the exact knot of a G^1 joint the right-sided convention of section 3 applies.

Wherever the parameter falls on a straight segment,

$$\kappa = 0$$

and the principal normal is mathematically undefined: raise `UndefinedPrincipalNormalError`. For a completely straight spline this holds for every parameter value.

Do not return an arbitrary normal and do not return a zero vector.

11.6 `curvature_vector(u)`

Return

$$\boxed{K(u) = \kappa(u) N_L(u).}$$

Its magnitude is

$$\|K(u)\| = |\kappa(u)|.$$

It is not a unit vector.

For a straight spline, return the zero vector.

11.7 Exact parallel offset as a NURBS

11.7.1 Public operation and sign convention

```
def offset(self, distance: Real) -> NURBSHandle: ...
```

`distance` is a finite signed length in user coordinates. Positive values use the left unit normal defined in Section 11.3 and negative values use the right unit normal. Therefore the required locus is

$$\boxed{r_d(u) = r(u) + d N_L(u), \qquad 0 \leq u \leq 1.}$$

This sign differs from references that define the positive normal as $(y', -x')/\|r'\|$. In particular, it is the negative of the normal used in Albrecht et al., Section 4.2. An implementation that follows that reference formula SHALL substitute $h = -d$.

The argument rules of Section 15.1 apply. A Boolean, array, sequence, NaN, or infinity is invalid. Zero is valid and follows the same construction; it MUST NOT select an undocumented reduced-degree representation.

11.7.2 Rational identity

For a regular span, let

$$v(t) = r'(t) = (x'(t), y'(t)), \qquad \sigma(t) = \|v(t)\| > 0,$$

and let $R_L(x, y) = (-y, x)$. The PH identity makes σ polynomial, so

$$r_d(t) = \frac{\sigma(t)r(t) + dR_L(v(t))}{\sigma(t)}$$

is rational for every finite d. This exact rationality, which an ordinary polynomial spline does not generally have, is a defining production feature of this PH spline family. Sampling, interpolation, least-squares fitting, and generic approximate offset algorithms are forbidden.

11.7.3 Homogeneous rational quintic controls

Let one source cubic span have degree-3 Bezier controls $C_0, \dots, C_3$ in its local parameter $t \in [0, 1]$. Its derivative and speed have degree-2 Bernstein forms

$$v(t) = \sum_{i=0}^{2} V_i B_i^2(t), \qquad V_i = 3(C_{i+1} - C_i),$$

$$\sigma(t) = \sum_{i=0}^{2} s_i B_i^2(t).$$

The s_i SHALL come from the stored PH speed polynomial and the V_i SHALL be independently checked against the stored hodograph. Define $q = 5$ and, for $k = 0, \dots, q$,

$$\lambda_{i,j}^{(k)} = \frac{\binom{2}{i}\binom{3}{j}}{\binom{5}{k}}, \qquad i + j = k.$$

The homogeneous offset controls $O_k = (W_k, X_k, Y_k)$ are uniquely defined by

$$\boxed{\begin{aligned} W_k &= \sum_{\substack{0 \le i \le 2 \\ 0 \le j \le 3 \\ i+j=k}} \lambda_{i,j}^{(k)} s_i, \\ (X_k, Y_k) &= \sum_{\substack{0 \le i \le 2 \\ 0 \le j \le 3 \\ i+j=k}} \lambda_{i,j}^{(k)} \left(s_i C_j + dR_L(V_i) \right). \end{aligned}}$$

The corresponding rational Bezier control point is $Q_k = (X_k/W_k, Y_k/W_k)$ with weight W_k. These are the degree-5 offset controls of Farouki, Section 17.5, written for this package's left-normal sign. The formula is a Bernstein product, not a fit.

The formula is coordinate-system independent, but C_j, V_i, s_i, and d SHALL use one common affine frame. The reference Python/NumPy profile performs the products in the normalized frame of Section 5 with $\hat{d} = d/H$, converts each verified Euclidean rational control back to user coordinates, and leaves the dimensionless weights unchanged. Mixing a physical distance with normalized controls is nonconforming.

11.7.4 Positive weights and canonical assembly

A positive denominator polynomial can have a nonpositive Bernstein coefficient on a wide interval. The public handle nevertheless requires strictly positive weights for stable standard NURBS interchange. For each homogeneous quintic patch, let ϵ be the arithmetic unit roundoff and

$$\gamma_n = \frac{n\epsilon}{1 - n\epsilon}, \qquad \tau_W = \gamma_{32(q+1)} \max_k |W_k|.$$

The reference binary64 profile has $q = 5$ and a maximum of 24 midpoint subdivision levels per source patch. The implementation SHALL:

1. accept a leaf only when every $W_k > \tau_W$;
2. if the test fails, subdivide all three homogeneous Bernstein coordinates by de Casteljau at the exact local midpoint $1/2$;
3. process the left child before the right child and repeat until every leaf passes;
4. raise `OffsetConstructionError` if the existing regularity bound cannot certify termination by level 24.

Regularity gives $\sigma(t) > 0$, so exact arithmetic guarantees termination. Subdivision changes only the representation, not the rational curve. An implementation in another arithmetic MAY use a different fixed depth and a proved tighter rounding bound, but it SHALL document both and preserve the same accept-or-fail rule; it MUST NOT accept a weight whose sign is unresolved.

The leaf patches SHALL then be concatenated in source traversal order. At a common endpoint, multiply every homogeneous control of the next patch by the one positive projective scale that makes the two endpoint triples equal. Verify the independently computed Euclidean endpoint before sharing it; a projective rescale MUST NOT hide a position or normal mismatch. After this step, omit the duplicate first control of every patch after the first.

For M final rational patches of degree $q = 5$ with strictly increasing global breakpoints

$$0 = \xi_0 < \xi_1 < \cdots < \xi_M = 1,$$

the canonical clamped knot vector is

$$\boxed{U = \{\xi_0^{[q+1]}, \xi_1^{[q]}, \ldots, \xi_{M-1}^{[q]}, \xi_M^{[q+1]}\}.}$$

Here $a^{[r]}$ means r consecutive copies of a. The output has degree 5, $5M + 1$ controls, and `num_spans == M`. Original source knots and any deterministic positivity-refinement knots appear at their exact normalized global parameters. No knot removal or degree reduction is permitted. This segmented form is the canonical repository representation, so the same source state and binary64 distance produce the same arrays.

11.7.5 Evaluation and verification

`NURBSHandle.point(u)` SHALL evaluate homogeneous controls $(W_k Q_{k,x}, W_k Q_{k,y}, W_k)$ by the standard de Boor algorithm and divide only once, after verifying a finite positive denominator. Its parameter is the source spline's unchanged normalized global parameter. Endpoint and knot selection follow Section 3.

Before publication, independently verify:

- the degree, control count, knot count, knot order, and endpoint multiplicities;
- finite controls and strictly positive weights;
- the Bernstein coefficient identities for $\sigma r + dR_L(v)$ and σ;
- every source and refinement breakpoint from both incident patches;
- at deterministic interior oracle parameters,

$$\text{offset}(d).\,\text{point}(u) = r(u) + dN_L(u)$$

within a degree- and scale-aware binary64 forward-error bound.

Failure raises `OffsetConstructionError`; an approximate or partially verified handle is never returned.

11.7.6 Cusps and self-intersections

Differentiation with respect to source arc length gives

$$\frac{dr_d}{ds} = (1 - d\kappa)T.$$

Thus the offset has a cusp where $1 - d\kappa = 0$ and can self-intersect for large $|d|$. These are properties of the exact parallel curve. Construction SHALL retain them and SHALL NOT reject, trim, smooth, or join them. They are not rational poles: the NURBS denominator is the strictly positive source speed and is independent of d. Trimming and cusp classification are outside the `NURBSHandle` interface.

12. Exact local arc length

For one segment,

$$\sigma(t) = At^2 + Bt + C.$$

The local arc length is exactly

$$\boxed{S(t) = \frac{A}{3}t^3 + \frac{B}{2}t^2 + Ct.}$$

The segment length is

$$L_i = S_i(1).$$

The implementation shall evaluate this using fused multiply-add operations when available:

$$S(t) = t\left(C + t\left(\frac{B}{2} + t\frac{A}{3}\right)\right).$$

When cancellation is significant, an equivalent completed-square form may be used.

Define

$$d = \text{Re}(\overline{a}b) = \frac{B}{2},$$

$$h = \frac{d}{A},$$

$$g = \frac{|\chi|}{A}.$$

Then

$$\sigma(t) = A\left((t+h)^2 + g^2\right),$$

and the cancellation-resistant arc-length expression is

$$\boxed{S(t) = At\left[\frac{(t+h)^2 + (t+h)h + h^2}{3} + g^2\right].}$$

Do not evaluate

$$(t+h)^3 - h^3$$

directly for small t.

The cubic-PH construction supports closed-form arc-length reparameterization, one of its central advantages. (reference)

13. Elementary local arc-length inverse

13.1 Uniqueness

For a regular segment,

$$\sigma(t) > 0 \qquad \text{on } [0,1].$$

Therefore

$$S'(t) = \sigma(t) > 0,$$

so S is strictly increasing.

For every

$$s \in [0, L_i],$$

there is exactly one root

$$t \in [0,1]$$

of

$$S(t) - s = 0.$$

The implementation shall compute this unique root directly. It shall not compute all roots of the cubic and then attempt to choose one.

In particular, use neither:

- `numpy.roots`;
- a companion-matrix eigenvalue calculation;
- a generic complex Cardano implementation.

13.2 Closed-form reduction

For $A > 0$, define

$$h = \frac{\mathrm{Re}(\overline{a}b)}{A},$$

$$g = \frac{|\,\mathrm{Im}(\overline{a}b)|}{A}.$$

Set

$$y = t + h.$$

Given target local length s, define

$$R = h^3 + 3g^2h + \frac{3s}{A}.$$

The inversion reduces to

$$\boxed{y^3 + 3g^2y = R.}$$

When $g > 0$, the derivative of the left side is

$$3(y^2 + g^2) > 0,$$

so there is one real root globally:

$$\boxed{y = 2g \sinh\left[\frac{1}{3}\,\mathrm{asinh}\left(\frac{R}{2g^3}\right)\right].}$$

Then

$$t = y - h.$$

When $g = 0$,

$$\boxed{y = \mathrm{cbrt}(R), \qquad t = y - h.}$$

When $A = 0$, the speed is constant:

$$\boxed{t = \frac{s}{C}.}$$

13.3 Cancellation-resistant recovery of t

When t is near zero, do not directly subtract h from y.

Using

$$y^3 + 3g^2y - (h^3 + 3g^2h) = \frac{3s}{A},$$

factor the left side:

$$(y - h)\left(y^2 + yh + h^2 + 3g^2\right) = \frac{3s}{A}.$$

Therefore compute

$$\boxed{t = \frac{3s/A}{y^2 + yh + h^2 + 3g^2}.}$$

This avoids catastrophic cancellation in $y - h$.

For a target closer to the segment end than its start, invert from the end.

Define

$$\tau = 1 - t,$$

use the reversed preimage

$$w_{\mathrm{rev}}(\tau) = w_1 - b\tau,$$

and invert the remaining length

$$L_i - s.$$

Finally compute

$$t = 1 - \tau.$$

This prevents loss of relative accuracy near $t = 1$.

13.4 Scaling the depressed cubic

The quantities h, g and R can become large when A is small.

Before evaluating the hyperbolic formula, scale the depressed cubic.

Choose

$$q = \max\left(g, \sqrt[3]{|R|}\right).$$

Set

$$Y = \frac{y}{q}, \qquad G = \frac{g}{q}, \qquad R_q = \frac{R}{q^3}.$$

Solve

$$Y^3 + 3G^2Y = R_q,$$

then recover

$$y = qY.$$

This keeps the principal dimensionless quantities near unit magnitude.

If computing

$$R_q/(2G^3)$$

would overflow or underflow, use a stable scaled Cardano form for the same unique real root. The smaller Cardano term must be obtained through the product identity rather than subtracting two nearly equal numbers.

For

$$Q = R_q/2, \qquad H_q = \operatorname{hypot}(Q, G^3),$$

use:

- if $Q \geq 0$,
$$U = \operatorname{cbrt}(Q + H_q), \qquad V = -\frac{G^2}{U};$$
- if $Q < 0$,
$$V = \operatorname{cbrt}(Q - H_q), \qquad U = -\frac{G^2}{V}.$$

Then

$$Y = U + V.$$

Near $R_q = 0$, prefer the hyperbolic form, because the Cardano sum may cancel.

13.5 Nearly constant-speed branch

If

$$A$$

is too small relative to B and C for the completed-square quantities to be well-conditioned, do not form $h = d/A$.

Use a lower-degree elementary initializer:

- linear initializer when B is also negligible;
- stable quadratic initializer when B is material;

then polish against the exact cubic $S(t) - s$.

This branch is a floating-point conditioning strategy, not a change to the stored curve.

13.6 Safeguarded polishing

The elementary result shall be followed by a bounded correction against the exact polynomial.

Maintain

$$[t_{\mathrm{lo}}, t_{\mathrm{hi}}] = [0, 1].$$

At each correction:

$$f(t) = S(t) - s,$$

$$f'(t) = \sigma(t) > 0.$$

Propose

$$t_N = t - \frac{f(t)}{\sigma(t)}.$$

Accept the Newton proposal only if:

- it is finite;
- it lies strictly inside the current bracket;
- it does not increase the residual excessively.

Otherwise use the bracket midpoint.

Normally one Newton correction should suffice after the elementary inverse. Permit up to three ordinary corrections, followed by a safeguarded Newton-bisection loop if needed.

The fallback shall have a fixed finite iteration bound, for example 64 iterations.

The stopping test must be based primarily on arc-length residual:

$$|S(t) - s| \le 64\varepsilon_{\mathrm{mach}} L_i + 4\,\mathrm{ulp}(s).$$

This has a direct geometric interpretation:

$$\|z(t) - z(t_*)\| \le |S(t) - s|.$$

If the residual requirement cannot be achieved, raise `ArcLengthInversionError`.

The algorithm shall never return a nonfinite value, an out-of-range root, or an unverified Newton iterate.

14. Global arc-length operations

14.1 Prefix lengths

Store normalized segment lengths

$$\widehat{L}_i$$

and prefix lengths

$$\widehat{C}_0 = 0,$$

$$\widehat{C}_{i+1} = \widehat{C}_i + \widehat{L}_i.$$

Use compensated summation.

Verify strict monotonicity:

$$\widehat{C}_{i+1} > \widehat{C}_i.$$

If floating-point resolution causes a prefix length not to increase, raise `LengthResolutionError`.

The total physical length is

$$L = H\widehat{C}_m.$$

14.2 `arc_length(u)`

For local segment parameter t,

$$\boxed{\text{arc\_length}(u) = H\left(\widehat{C}_i + \widehat{S}_i(t)\right).}$$

Required endpoint behavior:

$$\text{arc\_length}(0) = 0,$$

$$\text{arc\_length}(1) = L.$$

At a global knot u_i, return the stored prefix length directly.

14.3 `parameter_at_length(s)`

Domain:

$$s \in [0, L].$$

Required exact endpoint results:

$$\text{parameter\_at\_length}(0) = 0,$$

$$\text{parameter\_at\_length}(L) = 1.$$

Algorithm:

1. convert to normalized length:
$$\hat{s} = s/H;$$
2. locate the containing segment using binary search on prefix lengths;

3. set
$$s_{\text{local}} = \hat{s} - \widehat{C}_i;$$
4. compute the unique local inverse
$$t = \widehat{S}_i^{-1}(s_{\text{local}});$$
5. return
$$\boxed{u = \frac{i+t}{m}.}$$

At an exact prefix length, return the exact global knot i/m.

14.4 `point_at_length(s)`

Do not implement this as a public call to `parameter_at_length` followed by a second public call to `point`.

Instead:

1. validate s;
2. locate the segment once;
3. invert its local arc length once;
4. evaluate that segment directly.

This avoids duplicate binary searches and preserves exact interpolation at prefix lengths.

Because
$$\frac{dr}{ds} = T \qquad \text{and} \qquad \frac{d^2r}{ds^2} = \kappa N,$$
the regularity of the arc-length parametrization follows the continuity structure of section 10.3:

$$\boxed{s \mapsto \text{point\_at\_length}(s) \text{ is globally } C^1, \text{ and } C^2 \text{ on every convex sub-spline.}}$$

For strictly convex input data there are no G^1 joints and the map is globally C^2, as before. At the finitely many arc lengths of auxiliary inflection points and straight/curved transitions the first derivative T is continuous and the second derivative jumps by the curvature jump.

15. Argument validation

15.1 Parameter arguments

All scalar methods shall:

- accept Python real scalars and NumPy real scalar values;
- reject Boolean values;
- reject arrays and sequences;
- reject NaN and infinities.

For `u`:
$$0 \le u \le 1.$$

For `s`:
$$0 \le s \le L.$$

For `distance`, every finite real value, including zero and negative values, is valid. It is not restricted by the local radius of curvature. A value that would make an offset coordinate or homogeneous coefficient nonrepresentable raises `OffsetConstructionError` instead of returning infinity.

Values outside the domain produce an exception. Extrapolation is not supported.

A value lying at most a small number of ulps outside an endpoint because of prior floating-point arithmetic may be clamped to that endpoint. Larger violations must not be clamped silently.

15.2 Side argument

`normal(u, side)` accepts only:

```
"left"
"right"
```

Case conversion and aliases are not required.

16. Exception hierarchy

All package exceptions shall derive from `PHSplineError`. `CubicPHSplineError` and `PHBSplineError` are sibling family roots; neither shall inherit from the other. The value and runtime branches follow the same rule through neutral package bases:

```
PHSplineError
|-- PHSplineValueError, ValueError
|-- PHSplineRuntimeError, RuntimeError
|-- CubicPHSplineError
`-- PHBSplineError

CubicPHSplineValueError
    : CubicPHSplineError, PHSplineValueError
CubicPHSplineRuntimeError
    : CubicPHSplineError, PHSplineRuntimeError
PHBSplineValueError
    : PHBSplineError, PHSplineValueError
PHBSplineRuntimeError
    : PHBSplineError, PHSplineRuntimeError
```

Shared failures derive from both sibling concrete branches so either spline family root catches an error used by both implementations:

```
Shared value errors:
    InvalidPointDataError, InsufficientPointDataError,
    NonFiniteCoordinateError, DegeneratePointDataError,
    ParameterOutOfRangeError, ArcLengthOutOfRangeError,
    UndefinedPrincipalNormalError

Shared runtime errors:
    NonRegularSplineError, ArcLengthInversionError,
    LengthResolutionError, NumericalPrecisionError,
    OffsetConstructionError

Cubic-only value errors:
    NonSimplePointDataError, ReversalError,
    InterpolationDomainError
```

```
Cubic-only runtime errors:
    SplineConvergenceError, NonAdmissibleSegmentError,
    G2VerificationError
```

`NonConvexDataError` from earlier revisions of this specification is removed: nonconvex data is now interpolated (section 22), not rejected.

Every construction exception shall include:

- the relevant point or segment index;
- the failed quantity;
- the measured value;
- the required bound.

`OffsetConstructionError` SHALL additionally identify `operation="offset"`, the signed distance, and the positivity-refinement depth when applicable.

Example diagnostic content:

```
Interior turn-angle condition failed at point 7:
phi[6] + phi[7] = 4.1432 rad,
required < 4.0969092717 rad.
```

17. Required numerical prohibitions

The implementation shall not use:

- unconstrained Newton iteration for tangent-angle construction;
- an unbounded nonlinear optimizer;
- `numpy.roots` for arc-length inversion;
- generic complex cubic-root branch selection;
- finite-difference curvature;
- finite-difference tangent evaluation;
- raw normalization without checking the norm;
- direct subtraction $y - h$ near arc-length endpoints;
- direct subtraction of nearly equal cubic Cardano radicals;
- arbitrary replacement of a materially negative discriminant by zero;
- silent acceptance of a nearly cuspidal segment;
- $G^2 \to G^1$ degradation anywhere other than the auxiliary inflection points and straight/curved transitions of section 22;
- insertion of any point other than the one auxiliary inflection point per inflection span defined by section 22;
- sampled-normal, polyline, interpolation, or least-squares offset fitting;
- silent degree reduction, knot removal, cusp trimming, or self-intersection removal during exact offset construction;
- publication of a NURBS handle with a nonfinite control, a nonpositive weight, or an unverified homogeneous denominator;
- silent extrapolation.

18. Post-construction invariants

A successfully constructed object guarantees:

1. **One segment per input span, plus one per inflection span**

$$\text{segment count} = \text{len}(p) - 1 + n_{\text{infl}},$$

where n_{infl} is the number of auxiliary inflection points (section 22); $n_{\text{infl}} = 0$ for convex or collinear data.

2. **Interpolation**
$$r(i/m) = P_i.$$
3. **Regularity**
$$\sigma_i(t) > 0 \quad \forall i,\ t \in [0,1].$$
4. **PH identity**
$$z_i'(t) = w_i(t)^2.$$
5. **Geometric tangent continuity at every joint**
$$T_{i-1}(1) = T_i(0),$$
including all auxiliary inflection points and straight/curved transitions.
6. **Signed-curvature continuity at every G^2 joint**
$$\kappa_{i-1}(1) = \kappa_i(0),$$
and a verified curvature **sign change** at every auxiliary inflection point.
7. **G^2 geometry on every convex sub-spline; exact G^1 at the documented joints of section 22.** For convex input data this is global G^2.
8. **Globally C^1 arc-length parametrization, C^2 on every convex sub-spline** (globally C^2 for convex data).
9. **Strictly increasing arc length**
$$u_1 < u_2 \Longrightarrow S(u_1) < S(u_2).$$
10. **Elementary local inverse**, with safeguarded floating-point verification.
11. **Round-trip consistency**
$$\text{arc\_length}\left(\text{parameter\_at\_length}(s)\right) \approx s.$$
12. **Frame identities**
$$\|T\| = 1, \qquad \|N\| = 1, \qquad T \cdot N = 0.$$
13. **Curvature-vector identity**
$$K = \kappa N_L.$$
14. **Exact rational offsets.** For every finite signed distance whose binary64 result is representable, `offset(d)` returns a verified immutable rational quintic NURBS with the same global parameter and
$$r_d(u) = r(u) + dN_L(u).$$

19. Acceptance tests

19.1 Exact and near-exact geometry tests

The implementation test suite shall include:

- two-point straight line;
- several monotonically ordered collinear points;
- translated and uniformly scaled versions of the same data;
- clockwise and counterclockwise convex point sets;
- samples from a circle;
- samples from a parabola restricted to a convex section;
- highly nonuniform chord lengths within the permitted condition ratio;
- very small but nonzero curvature;

- angles close to, but safely below, the uniqueness bound;
- a single-inflection S sequence (one auxiliary point, verified G^1);
- alternating turn signs (one auxiliary point per inflection span);
- mixed straight and curved spans (straight sub-splines with verified G^1 transitions and zero curvature on the straight side);
- an inflection adjacent to a straight span (no auxiliary point: the sign change is absorbed by the straight sub-spline);
- free-form data whose distinct convex sub-polylines cross (accepted);
- determinism of the auxiliary point: identical input reproduces identical P', d' and knots bit-for-bit.

19.2 Invalid-data tests

Required failure cases:

- fewer than two points;
- malformed coordinates;
- Boolean coordinates;
- NaN or infinity;
- consecutive duplicates;
- nonconsecutive duplicates within one convex sub-polyline;
- collinear data with backtracking;
- a near-π reversal;
- a convex sub-polyline whose nonadjacent chords properly intersect;
- an interior uniqueness-angle violation inside a convex sub-spline;
- a prescribed-tangent boundary deviation violating the uniqueness bound (no clamping permitted there, section 7);
- coordinate range causing overflow during normalization.

19.3 Continuity tests

At every G^2 join verify independently:

$$\|T_- - T_+\| < 10^{-12},$$

and relative curvature mismatch below 10^{-10}.

At every auxiliary inflection point verify independently:

$$\|T_- - T_+\| < 10^{-12}, \qquad \kappa_- \, \kappa_+ < 0,$$

and that both one-sided tangents equal the prescribed direction d' of section 22 to within ε_T. At straight/curved transitions verify the tangent bound and exactly zero curvature on the straight side.

Evaluate from both segment sides, not only through the public right-sided knot convention.

19.4 Arc-length inversion tests

For each segment and for the complete spline:

- test $s = 0$;
- test $s = L_i$;
- test values within a few ulps of both endpoints;
- test random interior values;
- test monotonicity;
- test inversion from both traversal directions;
- test round trip $t \to S(t) \to t$;

- test round trip $s \to t \to S(t)$;
- verify that the returned root lies in $[0, 1]$;
- verify that no alternate root-selection logic is needed.

Required normalized arc-length residual:

$$|S(t) - s| \le 64\varepsilon_{\text{mach}} L_i + 4\,\text{ulp}(s).$$

19.5 Differential-frame tests

For random interior points verify:

$$|\|T\| - 1| < 64\varepsilon_{\text{mach}},$$

$$|\|N\| - 1| < 64\varepsilon_{\text{mach}},$$

$$|T \cdot N| < 64\varepsilon_{\text{mach}},$$

and

$$|\|K\| - |\kappa||$$

within relative floating-point tolerance.

For right-turning data verify:

$$\kappa < 0,$$

and that `principal_normal` points opposite the left normal.

For straight data verify that:

- signed curvature is zero;
- curvature vector is zero;
- `principal_normal` raises its specified exception.

19.6 Exact offset NURBS tests

For open and closed curves, and for straight, convex, inflectional, and mixed straight/curved data, test positive, negative, zero, near-zero, and large finite distances. Required checks are:

- exact public degree 5;
- `len(knots) == num_control_points + degree + 1`;
- clamped endpoint multiplicity 6 and internal multiplicity 5;
- nondecreasing finite knots, finite controls, and strictly positive weights;
- read-only snapshots that cannot mutate the handle;
- source and NURBS parameter domains both exactly $[0, 1]$;
- independent direct comparison with $r(u) + dN_L(u)$ at endpoints, every source/refinement knot from both sides, and dense random interior values;
- the signed identity $r_{-d}(u) = r(u) - dN_L(u)$;
- denominator and weights independent of `distance`, subject only to the same deterministic positivity refinement;
- deterministic repeated construction, including identical knot refinement;

- a cusp case satisfying $1 - d\kappa = 0$ within an independent oracle bound;
- a large-distance self-intersection case, with no trimming or rejection;
- malformed distance arguments and nonrepresentable output failures;
- direct high-precision verification of the homogeneous Bernstein coefficient formulas, not only sampled point agreement.

The offset acceptance oracle SHALL use at least 100 decimal digits on selected curved spans. A general-purpose approximate offset routine is not an oracle.

20. Suggested package organization

```
ph_spline/
    __init__.py
    base.py
    cubic.py
    segment.py
    construction.py
    nonlinear.py
    arclength.py
    nurbs.py
    predicates.py
    exceptions.py
    typing.py
    py.typed
```

Responsibilities:

- `base.py`: common abstract `PHSpline` geometry and distance interface;
- `cubic.py`: abstract `CubicPHSpline` family base, concrete `CubicPHSplineOpen` and `CubicPHSplineClosed`, and global parameter dispatch;
- `segment.py`: immutable cubic PH segment representation;
- `construction.py`: input classification, endpoint tangents, PH edge lengths;
- `nonlinear.py`: bounded tridiagonal G^2 solve;
- `arclength.py`: stable exact length and elementary inverse;
- `nurbs.py`: shared immutable `NURBSHandle`, homogeneous de Boor evaluation, exact PH offset products, positivity refinement, and assembly;
- `predicates.py`: robust orientation, collinearity and intersection predicates;
- `exceptions.py`: exception hierarchy;
- `typing.py`: public and internal type aliases.

The public namespace should expose the common base, the cubic implementation, and their documented exception types:

```
PHSpline
CubicPHSpline
CubicPHSplineOpen
CubicPHSplineClosed
NURBSHandle
PHSplineError
CubicPHSplineError
```

and the documented exception subclasses.

21. Final implementation contract

A call to

`curve = CubicPHSplineOpen(p)`

has exactly two valid outcomes:

1. it returns an immutable, regular planar PH spline satisfying all postconditions of section 18: one cubic per input span plus exactly one auxiliary segment pair per inflection span, G^2 on every convex sub-spline, and exact verified G^1 with a curvature sign change at every auxiliary inflection point and straight/curved transition - in particular, globally G^2 whenever the input data is convex - and the exact NURBS offset operation of Section 11.7; or
2. it raises a specific exception explaining why the requested spline cannot be constructed reliably.

It shall never return:

- a biarc spline;
- a spline with hidden or undocumented inserted points (auxiliary inflection points exist only where section 22 defines them, at deterministic, reproducible locations);
- a merely G^1 interpolant of convex data;
- a G^0 corner anywhere;
- a near-cusp;
- a nonconverged nonlinear approximation;
- an offset generated by sampling, fitting, or any other approximation;
- a curve whose arc-length inverse has not passed its residual check.

22. General data: auxiliary inflection points and G^1 joints

This section makes the constructor's treatment of general free-form data fully deterministic. It follows the preprocessing algorithm of the reference (Jaklič, Kozak, Krajnc, Vitrih, Žagar, Section 6): whenever the data polygon has an inflection, one additional point P' with a prescribed unit tangent d' is inserted on that span, the data is thereby split into convex sub-problems, and continuity at P' is reduced from G^2 to G^1. Where the reference leaves choices open ("choose d' appropriately"), this section fixes them.

22.1 Inflection spans

With the classification quantities of section 6, an **inflection span** is an input span $P_i \to P_{i+1}$ whose two bounding interior points carry material turns of opposite sign:

$$s_i\, s_{i+1} < 0, \qquad 1 \le i \le m-2.$$

Both neighbours P_{i-1} and P_{i+2} always exist. Exactly one auxiliary point is inserted per inflection span; their number is

$$n_{\text{infl}} = \#\{\, i : \; s_i\, s_{i+1} < 0 \,\}.$$

A sign change separated by a collinear sub-run is **not** an inflection span: the straight sub-spline itself carries the curvature through zero (section 22.5) and no point is inserted.

Let $\tau_L = s_i$ denote the orientation of the sub-sequence ending at P_i and $\tau_R = s_{i+1} = -\tau_L$ the orientation of the sub-sequence starting at P_{i+1}.

22.2 The insertion recipe

Following the reference, the local model is **the parametric cubic $C(\tau)$ interpolating the four surrounding points** $P_{i-1}, P_i, P_{i+1}, P_{i+2}$. The reference does not fix the parametrization; this specification prescribes chord-length knots

$$\tau_{i-1} = 0, \qquad \tau_{j+1} = \tau_j + \|P_{j+1} - P_j\|, \qquad j = i-1, i, i+1.$$

All quantities below are computed in the normalized coordinates of section 5.

Crossing parameter. Define the scalar cubic

$$w(\tau) = (C(\tau) - P_i) \times \Delta_i.$$

Since C interpolates, $w(\tau_i) = w(\tau_{i+1}) = 0$ exactly. Deflating these two known roots leaves a **linear** factor, whose root τ_* is unique and shall be computed in closed form from the cubic's coefficients. No iterative root finder and no root selection among multiple candidates is permitted.

Auxiliary point. If $\tau_* \in (\tau_i, \tau_{i+1})$, set the raw chord fraction

$$\rho_{\text{raw}} = \frac{(C(\tau_*) - P_i) \cdot \Delta_i}{\|\Delta_i\|^2},$$

then

$$\boxed{P' = P_i + \rho\,\Delta_i, \qquad \rho = \text{clamp}\big(\rho_{\text{raw}}, \tfrac{1}{16}, \tfrac{15}{16}\big)\,.}$$

P' lies **exactly on the chord**; the clamp is a conditioning guard that keeps both sub-chords at least 1/15 of each other.

Prescribed tangent. Set

$$d' = \frac{C'(\tau_*)}{\|C'(\tau_*)\|},$$

and let the **tilt** be the oriented angle from the chord to d':

$$\delta' = \tau_L \angle(\Delta_i,\, d') \quad \text{(signed)}.$$

Validity requires all of:

- forward orientation: $d' \cdot \Delta_i > 0$;
- crossing orientation: $\tau_L\,(\Delta_i \times d') > 0$, equivalently $\delta' > 0$ (at a transversal crossing of the chord this holds automatically: before τ_* the cubic lies on the $-\tau_L$ side, after it on the $+\tau_L$ side);
- a minimum tilt $\delta' \geq \delta_\theta$ (section 7.2 margin);
- the prescribed-boundary uniqueness conditions of the two adjacent sub-splines:

$$\delta' + \phi_i < \Theta_{\text{unique}} - \delta_\theta, \qquad \delta' + \phi_{i+1} < \Theta_{\text{unique}} - \delta_\theta.$$

Deterministic fallback. If the intersection is empty ($\tau_* \notin (\tau_i, \tau_{i+1})$), degenerate, or any validity condition fails, the reference prescribes the chord midpoint with an "appropriately" chosen direction; this specification fixes it as

$$\rho = \tfrac{1}{2}, \qquad \delta' = \tfrac{1}{2}\min(\phi_i, \phi_{i+1}, \tfrac{\pi}{2})\,, \qquad d' = Q(\tau_L\,\delta')\,\frac{\Delta_i}{\|\Delta_i\|},$$

where Q is the rotation matrix. The fallback always satisfies the uniqueness conditions: $\delta' \leq \pi/4$ and $\phi < \pi$ give $\delta' + \phi < 5\pi/4 \approx 3.927 < \Theta_{\text{unique}}$.

22.3 Exact G^1 and strictly positive deviations

Because P' lies on the chord, both sub-chords P_iP' and $P'P_{i+1}$ have the chord direction ψ_i. The boundary deviations of the two adjacent segments at P' are therefore both equal to the tilt:

$$\boxed{\beta_{\text{left}} = \tau_L(\theta' - \psi_i) = \delta', \qquad \alpha_{\text{right}} = \tau_R(\psi_i - \theta') = \delta' > 0.}$$

Both sub-splines share the identical unit tangent d' at the identical point P': the joint is exactly G^1 by construction, and the curvature changes sign across it because the two sub-splines have opposite orientations.

The strict positivity $\delta' > 0$ is not merely a numerical margin - the exact-chord tangent $\delta' = 0$ is **inadmissible**. Proof: a segment with end deviation $\beta = 0$ and start deviation $\alpha > 0$ has, in the notation of section 8, $\beta_0 = -\alpha/2$, $\beta_1 = \alpha/2$, hence $\xi_0 = -\frac{1}{2}$, $D = 1 + q$, $C_q = (3 + 2q)/4$, and $\Delta_q = D^2 - A_qC_q = \frac{1}{4}$ identically, so $\xi_1 = C_q/(D + \frac{1}{2}) = \frac{1}{2}$ and

$$\lambda_0 = \ell\,(\xi_1 + \xi_0) = 0 :$$

the segment degenerates to a curve with a stationary point at its start.

22.4 Sub-spline assembly

The auxiliary points, together with the straight/curved transition points of section 22.5, partition the data into convex sub-problems. Each curved sub-spline is solved by the machinery of sections 8-10 unchanged, with its boundary angles fixed beforehand:

- **free** boundaries (global spline ends): circumcircle policy with the minimal clamp (section 7);
- **prescribed** boundaries (auxiliary points, transitions): deviation fixed by this section; clamping is forbidden, violations raise `InterpolationDomainError`.

Every input turn ϕ_i is preserved exactly: the recipe adds only the two boundary deviations δ' at each auxiliary point and moves no input point. The nonlinear system decouples into independent tridiagonal blocks (section 9.3), each subject to the full acceptance battery of sections 9.6 and 10.

22.5 Straight sub-runs and transitions

Maximal collinear sub-runs form degenerate straight sub-splines (section 6.2). At each straight/curved transition input point the prescribed common tangent is the **line direction** of the straight sub-run, oriented along traversal. The adjacent curved sub-spline's boundary deviation there equals the material turn at the transition point, which is strictly positive by classification; the joint is exactly G^1 and the curvature jumps between 0 and a nonzero value.

A sign change whose bounding material turns are separated by a straight sub-run requires no auxiliary point: the curvature passes through zero along the straight segments.

22.6 Knots, arc length, evaluation

- The auxiliary point is a spline knot at $u_{P'} = (i + \rho)/m$ (section 3) and is interpolated exactly: $r(u_{P'}) = P'$.
- The public knot identity $r(i/m) = P_i$ is unchanged.
- Segment count, continuity and arc-length regularity are as stated in sections 18.1, 18.5-18.8: G^2 within convex sub-splines, exact G^1 with a curvature sign change at auxiliary points, globally C^1 arc-length parametrization.
- Arc length, its inverse and `point_at_length` treat auxiliary segments identically to ordinary segments (sections 12-14); prefix sums simply contain $m + n_{\text{infl}}$ segment lengths.

22.7 Determinism and diagnostics

The construction is bit-for-bit reproducible: identical input produces identical auxiliary points, tangents, knots and coefficients. For each inflection span the constructor shall record internally the span index, ρ, δ', and whether the fallback replaced the cubic-model recipe. Exceptions raised while processing an inflection span carry that span's index.

Confidence: high on the PH formulas, arc-length inverse, geometric API, admissible-data restrictions, and the G^1 insertion mechanics of section 22.3 (which follow the reference's Section 6); moderate-high on the prescribed point-only endpoint-tangent policy and on the deterministic completions of section 22.2 (chord-length parametrization, the conditioning clamp on ρ, and the fallback tilt), which are necessarily design choices at points the reference leaves open.

23. Closed cubic PH splines

23.1 Continuity contract and impossibility boundary

`CubicPHSplineClosed` SHALL interpolate a cyclic point list, close position exactly, and provide a verified oriented G^2 seam at $u = 0 \equiv 1$. Its continuity contract elsewhere is identical to the open class:

- G^2 at every join inside a same-sign curved run;
- G^1 at each section-22 auxiliary curvature-sign change;
- G^1 at a curved/straight transition;
- G^2 along a completely straight sub-run.

A globally G^2 sign-changing cubic PH loop is impossible. For a regular cubic PH segment $z' = w^2$ with $w = a + bt$,

$$\kappa(t) = \frac{2\,\mathrm{Im}(\bar{a}b)}{|a + bt|^4}.$$

The numerator is constant. If curvature vanishes at one regular endpoint, it vanishes on the whole segment and that segment is straight. Consequently, signed-curvature equality at a G^2 join propagates one nonzero curvature sign. This counterexample boundary SHALL be documented in the public API; an implementation MUST NOT claim global G^2 for a nonconvex closed loop.

23.2 Strictly convex cyclic system

For N authoritative cyclic points $P_0, \dots, P_{N-1}$ define chord i by $P_i \to P_{i+1 \bmod N}$. Every chord must be representable and nonzero. Let $\tau \in \{-1, +1\}$ be the common material turn sign, ψ_i consistently unwrapped chord angles, and $\phi_i > 0$ the turn from chord $i-1$ to chord i at point i.

There is one tangent fraction per cyclic point,

$$x_i \in (0, 1), \qquad \theta_i = \psi_{i-1} + \tau x_i \phi_i,$$

with indices understood cyclically and angles unwrapped. Segment i has the positive endpoint deviations

$$\alpha_i = (1 - x_i)\phi_i, \qquad \beta_i = x_{i+1}\phi_{i+1}.$$

The section-8 elementary formulas SHALL compute its two positive PH edge lengths. Let $k_i^{(0)}$ and $k_i^{(1)}$ be the positive curvature magnitudes at its start and end. The cyclic residual is

$$F_i(x) = \log k_{i-1}^{(1)} - \log k_i^{(0)}, \qquad i = 0, \dots, N-1.$$

Thus the system has exactly N real tangent unknowns and N curvature equations. Tangent continuity is structural because both adjacent segments share θ_i. No open endpoint tangent convention remains and no coefficient or derivative may be assigned zero to close the count.

This square count generically removes continuous geometric freedom, but is not by itself an existence or uniqueness theorem: the nonlinear system can have no admissible root, one root, or several isolated roots. The reference profile SHALL use deterministic centered-secant and chord-weighted starts, select the first strictly accepted solution, and raise a typed convergence or admissibility error when no start produces a verified root.

23.3 Cyclic numerical solve

Each residual depends only on x_{i-1}, x_i, x_{i+1}, so the Jacobian is cyclic tridiagonal. The reference implementation SHALL:

1. evaluate the guarded residual only with elementary cubic-PH expressions;
2. assemble its Jacobian by complex-step differentiation using a deterministic distance-two coloring of the cyclic columns (at most five colors);
3. globalize with the bounded trust-region least-squares solve on $[x_{\min}, 1 - x_{\min}]^N$;
4. apply a damped cyclic Newton polish;
5. rebuild with unguarded formulas and accept only when $\max_i |F_i| \le F_{\text{tol}}$;
6. independently verify reconstruction, regularity, oriented convexity, position closure, unit-tangent equality and signed-curvature equality at all N joins.

For a simple convex loop the unwrapped tangent turns by $2\pi\tau$. Therefore the last endpoint preimage uses the antiperiodic lift

$$w(1) = -w(0),$$

while $w(1)^2 = w(0)^2$. This sign is a discrete square-root gauge and does not constitute geometric shape freedom. The public guarantee is geometric G^2; it does not promise equality of the two local segment speeds or a globally C^2 piecewise-polynomial parameterization.

23.4 General cyclic data and section-22 reuse

When cyclic turns change sign or contain straight classifications, the implementation SHALL reuse section 22 rather than invent a second inflection policy. The reference construction forms five exact repetitions of the cyclic point list plus the closing first point, runs the unchanged open planner, and publishes only the central period. Two guard periods on each side ensure that every central convex block has prescribed auxiliary boundaries and is independent of the artificial outer open boundaries.

The crop SHALL:

- locate the exact central user knots $2/5$ and $3/5$;
- copy and reindex every compiled segment in that period;
- remap its knots affinely to $[0, 1]$ and snap each authoritative knot exactly to i/N after a bounded residual check;
- translate auxiliary span indices back to $0, \dots, N-1$;
- copy the declared join kinds and prescribed tangents, including the seam;
- require the seam kind to be `g2`;
- compare the following guarded period's Bézier nets with the published period and reject a nonperiodic result;
- rerun regularity, admissibility and cyclic continuity verification on the cropped segments.

This repeated construction is a deterministic way to apply the existing local subsegment-inflection machinery to cyclic indexing. It is not an approximation and it MUST NOT expose the guard periods as public points or segments.

23.5 Closed parameter and distance behavior

The authoritative knot identity is

$$r(i/N) = P_i, \qquad i = 0, \ldots, N-1,$$

and $r(1) = r(0) = P_0$. `arc_length`, `parameter_at_length` and `point_at_length` retain the finite prefix domains $u \in [0, 1]$ and $s \in [0, L]$; they do not implicitly wrap out-of-range scalar arguments. At both domain ends, `point`, `tangent`, `normal` and `signed_curvature` SHALL return seam-consistent values within their documented tolerances. `offset(d)` SHALL return a clamped NURBS on the same finite parameter domain; its values at 0 and 1 are the same seam offset point. It does not expose the five-period construction guards.

23.6 Required closed tests

Conformance tests SHALL include:

- clockwise and counterclockwise cyclic polygons with 3, 4, 5, 8 and at least 64 points;
- exact interpolation at every i/N;
- independent position, tangent and signed-curvature seam checks;
- deterministic repeated construction;
- nonconvex radial waves and symmetric zigzag stars exercising multiple auxiliary subsegment inflections;
- rejection of fewer than three points and a repeated final seam point;
- rejection when the selected seam is a curved/straight G^1 transition;
- arc-length inversion and endpoint distance identity;
- exact positive- and negative-distance NURBS offsets with equal values at both seam parameters;
- the complete 128-image `examples/cubic_closed` generation run.

24. References

1. M. Jaklič, J. Kozak, M. Krajnc, V. Vitrih, and E. Žagar, "On interpolation by planar cubic G^2 Pythagorean-hodograph spline curves", *Mathematics of Computation* 79 (2010), 305-326, DOI 10.1090/S0025-5718-09-02298-4.
2. R. T. Farouki, *Pythagorean-Hodograph Curves: Algebra and Geometry Inseparable*, Springer, 2008, Section 17.5, especially the general homogeneous offset formula and the rational quintic controls for PH cubics.
3. L. Piegl and W. Tiller, *The NURBS Book*, second edition, Springer, 1997, for the NURBS definition, homogeneous de Boor evaluation, knot insertion, and rational Bezier decomposition.